

SE - 801 – Software Project Lab III

Midterm Technical Report

CodePulse

Intelligent Multi-Platform Coding Performance Analyzer

Submitted by:

Abdus Salam Islam Badhon

ID: BSSE-1401

Institute of Information Technology
University of Dhaka

Supervised by:

Mohd. Zulfiquar Hafiz

Professor, Department of IIT
Institute of Information Technology
University of Dhaka

Date: June 2, 2026



Table of Contents

1. Project Overview.....	3
1.1 Project Title.....	3
1.2 Problem Statement.....	3
1.3 Objectives.....	4
1.4 Scope.....	4
1.5 Deliverables.....	5
2. Requirements Analysis.....	6
2.1 Functional Requirements.....	6
2.1.1 Authentication & User Management.....	6
2.1.2 Data Synchronisation.....	6
2.1.3 Analytics.....	6
2.1.4 Recommendations.....	7
2.1.5 AI Coach.....	7
2.2 Non-Functional Requirements.....	7
2.3 Stakeholders.....	8
3. System Modeling.....	8
3.1 Use Case Modeling & Descriptions.....	8
3.1.1 System Use Case Diagram.....	8
Figure 1 : Level-0 Use Case Diagram.....	10
3.1.2.1 UC-1 Use Case Diagram: Authentication & Profile Handle Configuration.....	11
Figure 2 : Use Case Diagram — User Authentication & Profile Configuration Pipeline.	12
3.1.2.2 UC-2 Use Case Diagram: Multi-Platform Data Sync & Pruning.....	12
Figure 3 : Use Case Diagram — Multi-Platform Data Sync & Pruning.....	13
3.1.2.3 UC-3 Use Case Diagram: Analytics Computation & Hidden Insights Engine.....	13
Figure 4 : Use Case Diagram — Analytics Computation & Hidden Insights Engine..	14
3.1.2.4 UC-4 Use Case Diagram: Algorithmic Recommendation Generation & Feedback...	15
Figure 5 : Use Case Diagram — Algorithmic Recommendation Generation & Feedback.....	16
3.1.2.5 UC-5 Use Case Diagram: AI Coach Consultation & Session Persistence.....	16
Figure 6 : Use Case Diagram — AI Coach Consultation & Session Persistence.....	17
3.3 Activity Diagram.....	17
3.2.1 AD-1: User Authentication & Profile Configuration Pipeline.....	18
Figure 10 : Activity Diagram —Algorithmic Recommendation & Feedback Pipeline.	22
Figure 11 : Activity Diagram —AI Coach Prompt-to-Recommendation Pipeline.....	23
4.1 Database Schema.....	24
4.2 ER Diagram.....	25
Figure 12 : ER Diagram.....	26
5. Software Design.....	26

5.1	Component Diagram.....	26
	Figure 13 : Component Diagram.....	27
5.2	Key Design Decisions.....	28
5.2.1	Direct HTTP vs Spring AI SDK for Mistral.....	28
5.2.2	CodePulse Rating Algorithm.....	28
5.2.3	Submission Pruning.....	28
5.2.4	Hidden Insights Engine.....	28
6.	AI Engineering Design.....	29
6.1	AI Pipeline.....	29
6.2	Model Selection.....	31
6.3	Prompt Design.....	31
7.	Preliminary Test Plan.....	31
7.1	Testing Objectives.....	31
7.2	Features to Be Tested.....	31
7.3	Test Cases.....	32
8.	Timeline.....	34
8.1	Development Phases.....	34
8.2	Gantt Chart.....	35
	Figure 15 : Gantt Chart (Weeks 1–16).....	35
9.	Appendix.....	34
9.A	API Endpoint Reference.....	34
9.B	Project Source Code Link.....	35

1. Project Overview

1.1 Project Title

CodePulse — Intelligent Multi-Platform Coding Performance Analyzer

1.2 Problem Statement

Competitive programmers and software engineering students practise across multiple platforms — Codeforces, LeetCode, AtCoder, and CodeChef. Each platform operates in isolation, presenting only its own statistics and offering no consolidated view of a user's overall performance. As a result, learners face several concrete challenges:

- No single place to see combined solved problems, acceptance rates, or activity trends across all four platforms.
- No automated mechanism to detect topic weaknesses that span platforms (e.g., failing Dynamic Programming on both CF and LC).
- Problem selection for future practice is ad-hoc — users cannot state their goals in natural language and receive personalised recommendations.

- There is no unified rating that reflects competency across all four platforms simultaneously.
- Behavioural patterns such as declining activity, comfort-zone stagnation, or streak loss are invisible without manual log-keeping.

CodePulse solves all of these by acting as an analytical intelligence layer above the four platforms.

1.3 Objectives

- Build a full-stack web application that aggregates competitive programming data from Codeforces, LeetCode, AtCoder, and CodeChef into a single dashboard.
- Compute a unified CodePulse Rating from cross-platform activity using a weighted Elo-like algorithm.
- Implement automated topic-level weakness detection and a curated problem recommendation engine.
- Integrate Mistral AI (LLM) to accept free-text coaching prompts from users and return personalised problem recommendations grounded in the user's actual performance history.
- Generate hidden analytical insights — automatically detected patterns the user would not otherwise notice.
- Implement scheduled and on-demand synchronisation with all four platform APIs, with a configurable submission ceiling (default: 300 per user) to keep the system performant.

1.4 Scope

In scope:

- Authentication, user profile management with 4 platform handles.
- Data synchronisation with Codeforces REST API, LeetCode GraphQL API, AtCoder kenkoooo.com community API, and CodeChef community wrapper API.
- Combined analytics: CodePulse Rating, streak tracking, difficulty distribution, topic coverage, weakness radar, activity heatmap, per-platform breakdown, hidden insights engine.
- Curated (algorithmic) recommendations across all four platforms.

- AI Coach: Mistral-powered prompt-to-recommendation with full context injection and DB-persisted sessions.
- Automatic 6-hour sync schedule and manual sync option.
- Submission pruning to keep the latest 300 per user.

Out of scope: contest scheduling, real-time editorial fetching, social/collaborative features, mobile native app.

1.5 Deliverables

- Full-stack application: Spring Boot 3 backend + Next.js 14 frontend.
- PostgreSQL schema with 7 tables (users, problems, topics, submissions, recommendations, ai_sessions, ai_items).
- REST API with 20+ documented endpoints.
- Technical report (this document).

2. Requirements Analysis

2.1 Functional Requirements

2.1.1 Authentication & User Management

- FR-1: Users can register with username, email, and password. Passwords are BCrypt-hashed.
- FR-2: Users can log in and receive a JWT access token (24-hour expiry).
- FR-3: Users can store and update their Codeforces, LeetCode, AtCoder, and CodeChef handles via their profile.
- FR-4: Platform handles are always fetched fresh from the database on profile and dashboard load.

2.1.2 Data Synchronisation

- FR-5: The system fetches up to 200 recent submissions per sync from Codeforces (/user.status), LeetCode (GraphQL), AtCoder (kenkoooo API), and CodeChef (community API).
- FR-6: Submissions are deduplicated by platformSubmissionId before storage.
- FR-7: If a user's total submission count exceeds 300, the oldest submissions are automatically pruned, keeping only the most recent 300.
- FR-8: Synchronisation runs automatically every 6 hours via a scheduled cron job and can also be triggered manually by the user.

2.1.3 Analytics

- FR-9: The system computes a combined CodePulse Rating from all connected platforms using a weighted average algorithm.
- FR-10: Current streak, longest streak, and total active days are calculated across all platforms combined.
- FR-11: Topic-level weakness scores (failure rate per topic) are computed from all submissions.
- FR-12: The system generates 6–10 hidden insights automatically from the user's data (weakness detection, comfort-zone analysis, streak pattern, activity trend, platform imbalance, etc.).

- FR-13: Per-platform breakdown stats (solved count, acceptance rate, verdict distribution, topic coverage) are returned alongside combined stats.

2.1.4 Recommendations

- FR-14: The algorithmic recommendation engine selects up to 10 unsolved problems from the user's weakest topics within their estimated difficulty range.
- FR-15: Generating new recommendations replaces all previous ones for that user.
- FR-16: Users can mark recommendations as solved (feedback loop) or dismiss them.

2.1.5 AI Coach

- FR-17: Users can submit a free-text prompt describing their practice goal.
- FR-18: The system builds a rich context payload (rating, weakness scores, per-platform stats, recently solved problems, recent WA/TLE topics) and sends it to Mistral AI alongside the user's prompt.
- FR-19: Mistral returns a structured JSON response with problem recommendations, a coach insight paragraph, and focus areas.
- FR-20: AI recommendation sessions are persisted to the database; the latest session is shown to the user on page load without requiring regeneration.
- FR-21: Regenerating AI recommendations deactivates the previous session and creates a new one.
- FR-22: Users can mark AI-recommended items as solved or dismiss them.

2.2 Non-Functional Requirements

- NFR-1 Performance: API responses for analytics must be returned within 2 seconds for a user with up to 300 submissions.
- NFR-2 Security: All endpoints except `/api/auth/**` are protected by JWT. Passwords are never stored in plaintext.
- NFR-3 Reliability: The background sync service catches and logs failures per-platform without crashing the overall job.
- NFR-4 Scalability: Submission pruning ensures database growth is bounded per user. Async `@Scheduled` syncs prevent blocking the request thread.

- NFR-5 Usability: The frontend provides real-time loading skeletons, toast notifications, and empty states with action prompts.
- NFR-6 Maintainability: Each platform sync is an independent service class; adding a new platform requires adding only one new service and registering its handle in the User entity.
- NFR-7 AI Correctness: The LLM prompt is structured to produce only valid JSON. Responses are parsed via JsonNode (not class mapping) to tolerate future API schema additions without code changes.

2.3 Stakeholders

Stakeholder	Role	Interest in CodePulse
Competitive Programmer	Primary user	Improve problem-solving skills efficiently
CS Student	Primary user	Prepare for coding interviews and contests
Supervisor / Evaluator	Academic oversight	Assess technical depth and research contribution
Developer / Maintainer	Builder	Clean architecture, extensible platform

3. System Modeling

System modeling provides both structural and behavioral representations of CodePulse. This section details the system's operational scope using Use Case modeling alongside fine-grained Activity Diagrams that detail execution logic across the Spring Boot backend, PostgreSQL persistence layer, and external integrations.

3.1 Use Case Modeling & Descriptions

3.1.1 System Use Case Diagram

The Level-0 System Use Case Diagram illustrates the interactions between primary human actors, automated system schedulers, internal system boundaries, and external third-party API services.

Primary use cases:

- Register / Login — Create account and authenticate with JWT.
- Manage Profile — Add or update platform handles; view connection status.
- Sync Data — Trigger cross-platform submission fetch (manual or automated).
- View Dashboard — See CodePulse Rating, streak, platform connections, recent submissions.
- View Analytics — Access deep performance charts, platform breakdown, and hidden insights.
- Get Curated Recommendations — Browse algorithmically selected problems; mark solved or dismiss them.
- Ask AI Coach — Enter a free-text prompt; receive AI-generated problem recommendations grounded in personal history.

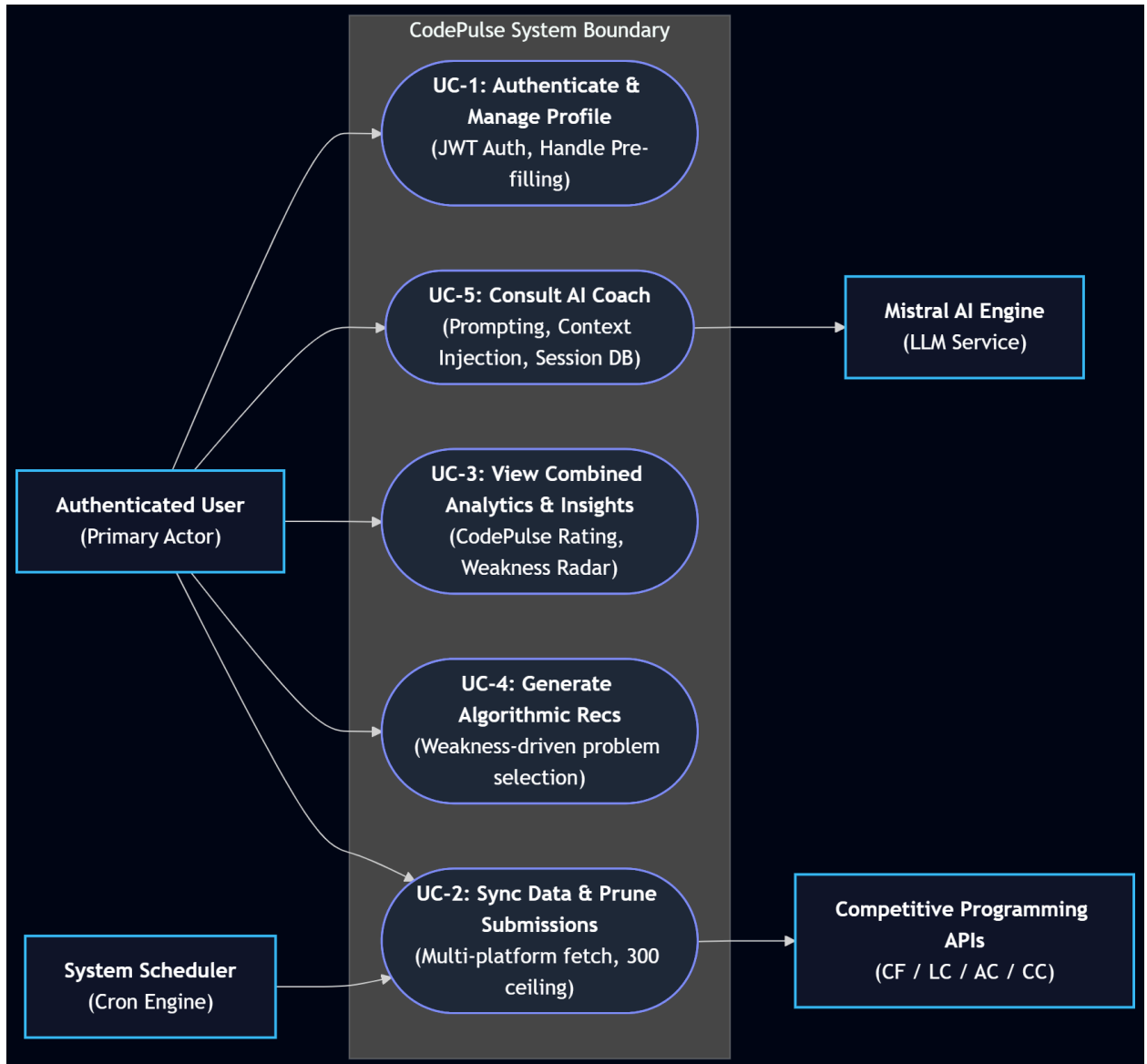


Figure 1 : Level-0 Use Case Diagram

3.1.2 Detailed Use Case Descriptions

3.1.2.1 UC-1 Use Case Diagram: Authentication & Profile Handle Configuration

Field	Description
Actor	Authenticated User
Trigger	User navigates to Register/Login page or updates profile settings (/profile).
Preconditions	System backend is operational; PostgreSQL database connection is active.
Main Flow	1. User registers with email, username, and password, or logs in with credentials.2. Backend authenticates user via BCrypt verification and issues a 24-hour signed JWT access token.3. User enters or updates platform handles (Codeforces, LeetCode, AtCoder, CodeChef) in Profile.4. Backend pre-fills and persists handles in the users table.5. System triggers an initial asynchronous sync for new handles.
Post-conditions	JWT stored in client state; platform handles saved and ready for cross-platform data fetching.
Exceptions	Invalid credentials → Return 401 Unauthorized.Duplicate email/username → Return 409 Conflict.

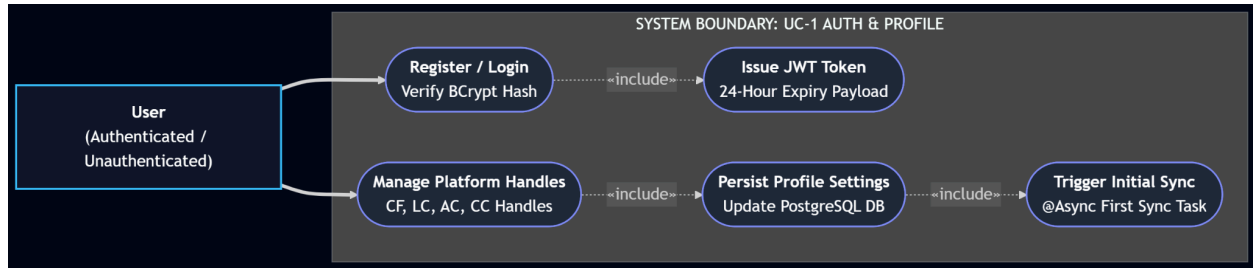


Figure 2 : Use Case Diagram — User Authentication & Profile Configuration Pipeline

3.1.2.2 UC-2 Use Case Diagram: Multi-Platform Data Sync & Pruning

Field	Description
Actor	Authenticated User / Background Cron Scheduler (<code>ScheduledSyncService</code>)
Trigger	User clicks "Sync All Platforms" (<code>POST /api/analytics/sync</code>) or automated 6-hour cron fires.
Preconditions	User profile contains at least one connected competitive programming handle.
Main Flow	1. System initiates parallel <code>@Async</code> requests to connected platform APIs (Codeforces REST, LeetCode GraphQL, AtCoder Kenkoooo, CodeChef API). 2. System fetches up to 200 recent submissions per platform. 3. Incoming submissions are deduplicated using <code>platformSubmissionId</code>. 4. New <code>Problem</code> and <code>Topic</code> records are generated and saved if non-existent. 5. <code>SubmissionPruningService</code> checks user total submission count; if count > 300, the oldest submissions are deleted to maintain a 300 ceiling. 6. <code>lastSyncedAt</code> timestamp is updated in the <code>users</code> table.

Post-conditions	Database contains up to 300 recent submissions per user; analytics are recalculated on demand.
Exceptions	Individual platform API timeout/rate limit → Log error for failing platform, continue syncing remaining platforms.

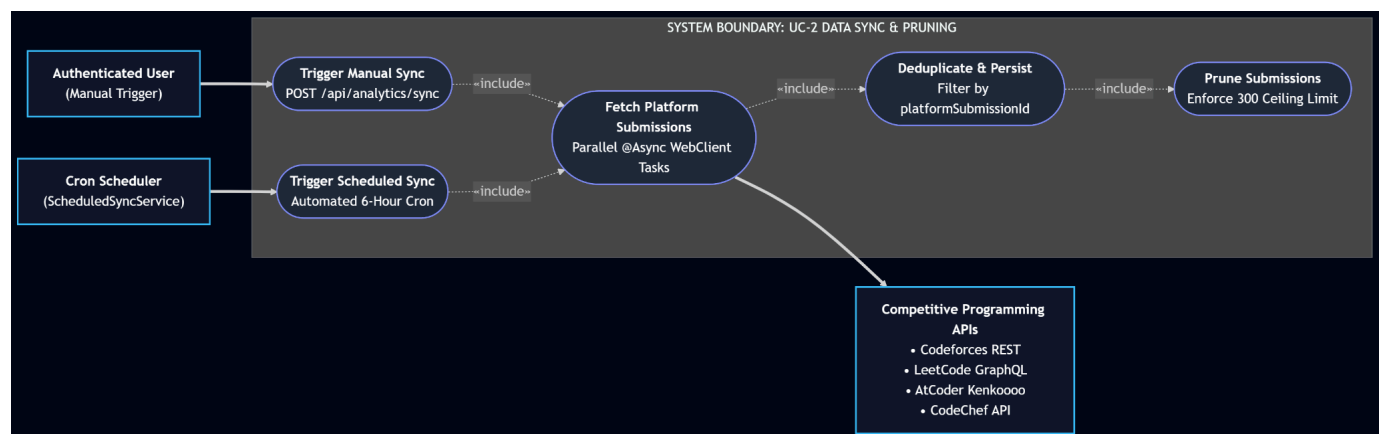


Figure 3 : Use Case Diagram — Multi-Platform Data Sync & Pruning

3.1.2.3 UC-3 Use Case Diagram: Analytics Computation & Hidden Insights Engine

Field	Description
Actor	Authenticated User
Trigger	User views Dashboard or Analytics page (GET /api/analytics/me).
Preconditions	User is authenticated with a valid JWT token; user has synced submission data.

Main Flow	1. AnalyticsServiceImpl fetches user submissions and connects to CombinedRatingCalculator.2. System calculates individual platform ratings and computes weighted Elo CodePulse Rating.3. System calculates active streak, longest streak, and combined active days.4. System evaluates topic failure rates to map strength/weakness distributions.5. Hidden Insights Engine analyzes data array to generate 6–10 automated findings (e.g., comfort-zone stagnation, platform imbalance).6. Return combined PerformanceAnalyticsResponse DTO to Next.js frontend.
Post-conditions	Analytics charts, CodePulse Rating, streak stats, and hidden insights rendered on UI.
Exceptions	No submission history available → Return zeroed stats DTO with empty state UI flags.

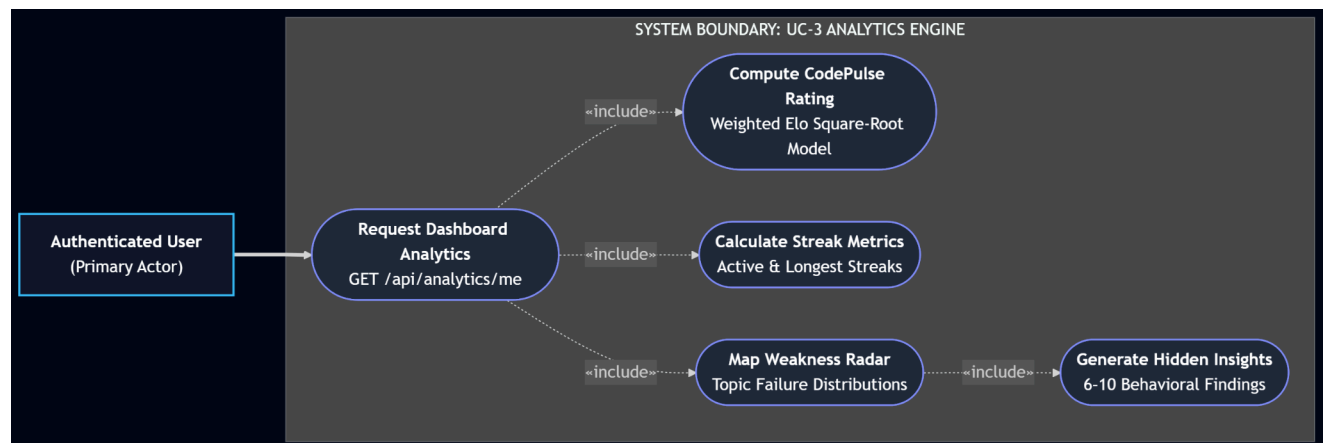


Figure 4 : Use Case Diagram — Analytics Computation & Hidden Insights Engine

3.1.2.4 UC-4 Use Case Diagram: Algorithmic Recommendation Generation & Feedback

Field	Description
Actor	Authenticated User
Trigger	User requests curated recommendations (<code>POST /api/recommendations/generate</code>).
Preconditions	User has weakness scores calculated from synced submission history.
Main Flow	1. <code>RecommendationServiceImpl</code> identifies user's top weakness topics from problem submission failure rates. 2. Engine determines user's difficulty rating bracket based on CodePulse Rating. 3. Engine queries <code>problems</code> database for unsolved problems in those weakness topics within rating bounds. 4. System selects up to 10 curated problems across all 4 platforms. 5. System deletes previous recommendations for user and persists new items in <code>recommendations</code> table. 6. User interacts with recommendations by marking them as solved (<code>PATCH /{id}/solved</code>) or dismissing them (<code>PATCH /{id}/dismiss</code>).
Post-conditions	Curated recommendation list updated; solved/dismissed flags persisted in database.
Exceptions	Insufficient database problem catalog → Fall back to standard difficulty-matched platform problems.

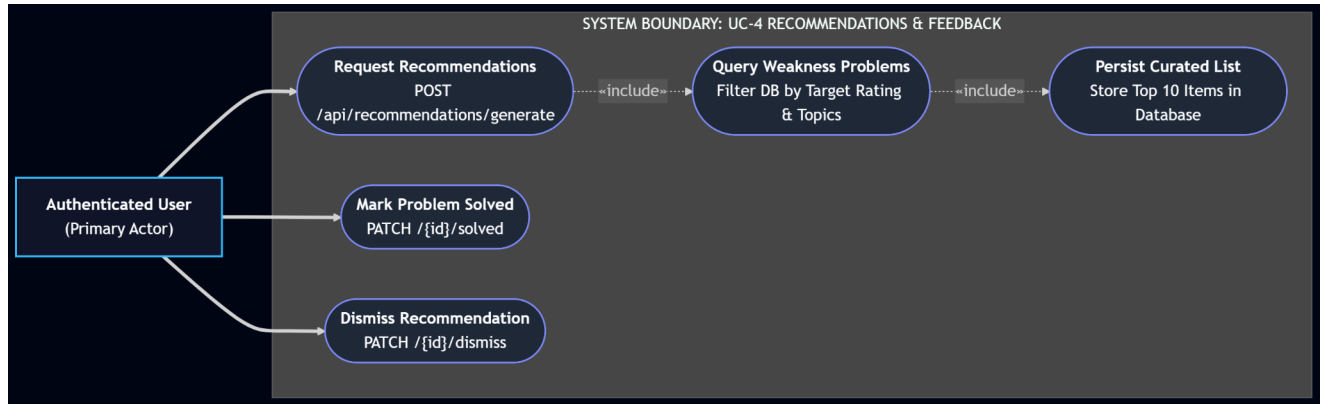


Figure 5 : Use Case Diagram — Algorithmic Recommendation Generation & Feedback

3.1.2.5 UC-5 Use Case Diagram: AI Coach Consultation & Session Persistence

Field	Description
Actor	Authenticated User
Trigger	User submits free-text practice prompt (POST /api/ai/recommend).
Preconditions	Mistral AI API key configured (spring.ai.mistralai.api-key); user is logged in.

Main Flow	1. System triggers context assembly: pulls CodePulse Rating, per-platform stats, top 8 weak topics, top 5 strong topics, and recent WA/TLE struggle topics. 2. Prompt Assembler merges user prompt with context payload and strict system JSON schema. 3. Backend executes POST request to Mistral AI endpoint (<i>mistral-large-latest</i>). 4. <i>JsonNode</i> parser validates structural integrity of response. 5. Active status of previous session in <i>ai_recommendation_sessions</i> is set to <i>false</i>. 6. System saves new session and items to <i>ai_recommendation_sessions</i> and <i>ai_recommendation_items</i>. 7. Return structured AI Coach recommendations, coach insight, and focus areas to frontend.
Post-conditions	DB stores session history; latest AI Coach session auto-loads on page render.
Exceptions	Malformed JSON response or Mistral API error → Catch exception, log error, execute <i>buildError()</i> fallback, retain previous session.

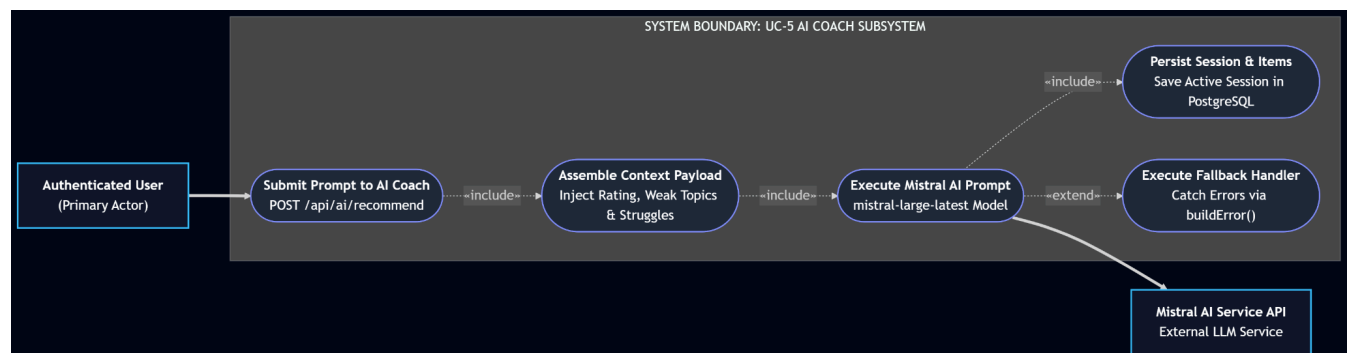


Figure 6 : Use Case Diagram — AI Coach Consultation & Session Persistence

3.3 Activity Diagram

Activity diagrams model the detailed operational flows, branching decisions, and state transitions for CodePulse's core execution pipelines.

3.2.1 AD-1: User Authentication & Profile Configuration Pipeline

This flow captures initial user registration, BCrypt password hashing, JWT token issuance, and profile handle persistence.

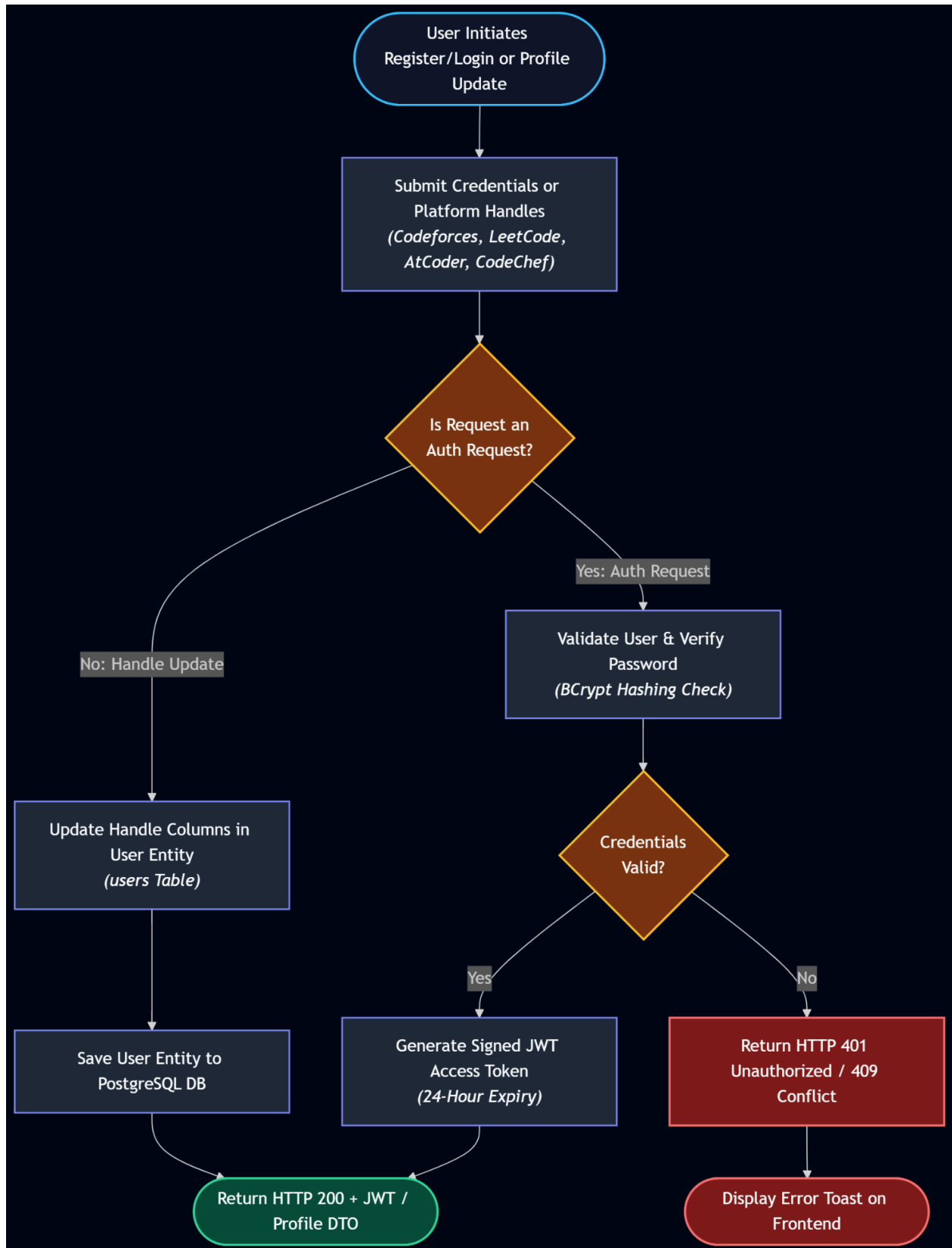


Figure 7 : Activity Diagram — User Authentication & Profile Configuration Pipeline

3.2.2 AD-2: Multi-Platform Async Synchronisation & Pruning Pipeline

This flow captures asynchronous multi-platform data fetching, submission deduplication, database storage, and the 300-submission ceiling pruning algorithm.

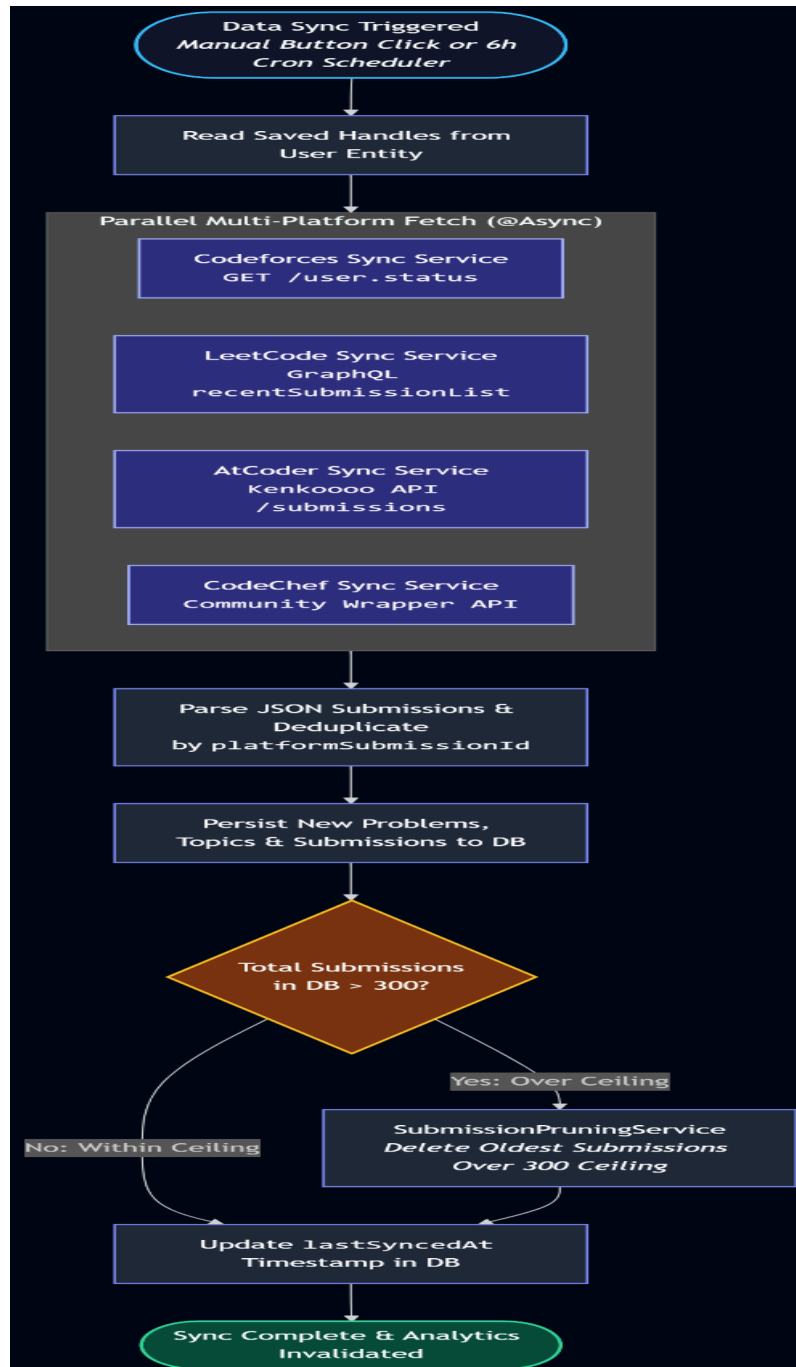


Figure 8 : Activity Diagram —Multi-Platform Async Synchronisation & Pruning Pipeline

3.2.3 AD-3: Analytics Engine & CodePulse Rating Pipeline

This flow details how analytics requests aggregate cross-platform data, compute weighted Elo ratings, calculate active streaks, and trigger the Hidden Insights Engine.

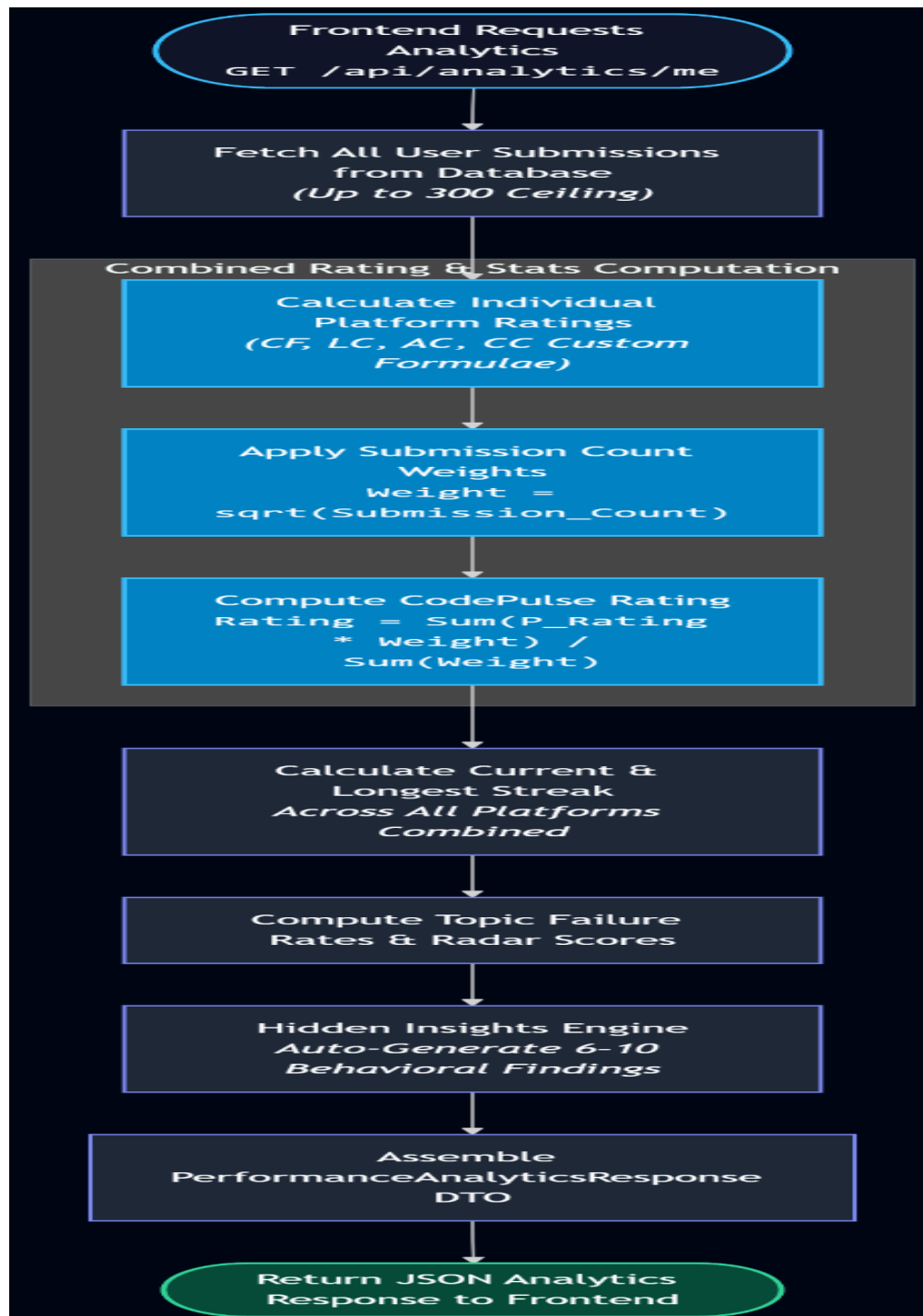


Figure 9 : Activity Diagram —Analytics Engine & CodePulse Rating Pipeline

3.2.4 AD-4: Algorithmic Recommendation & Feedback Pipeline

This flow illustrates weak-topic detection, rating-bracket problem querying, recommendation invalidation, and user feedback tracking (solving/dismissing).

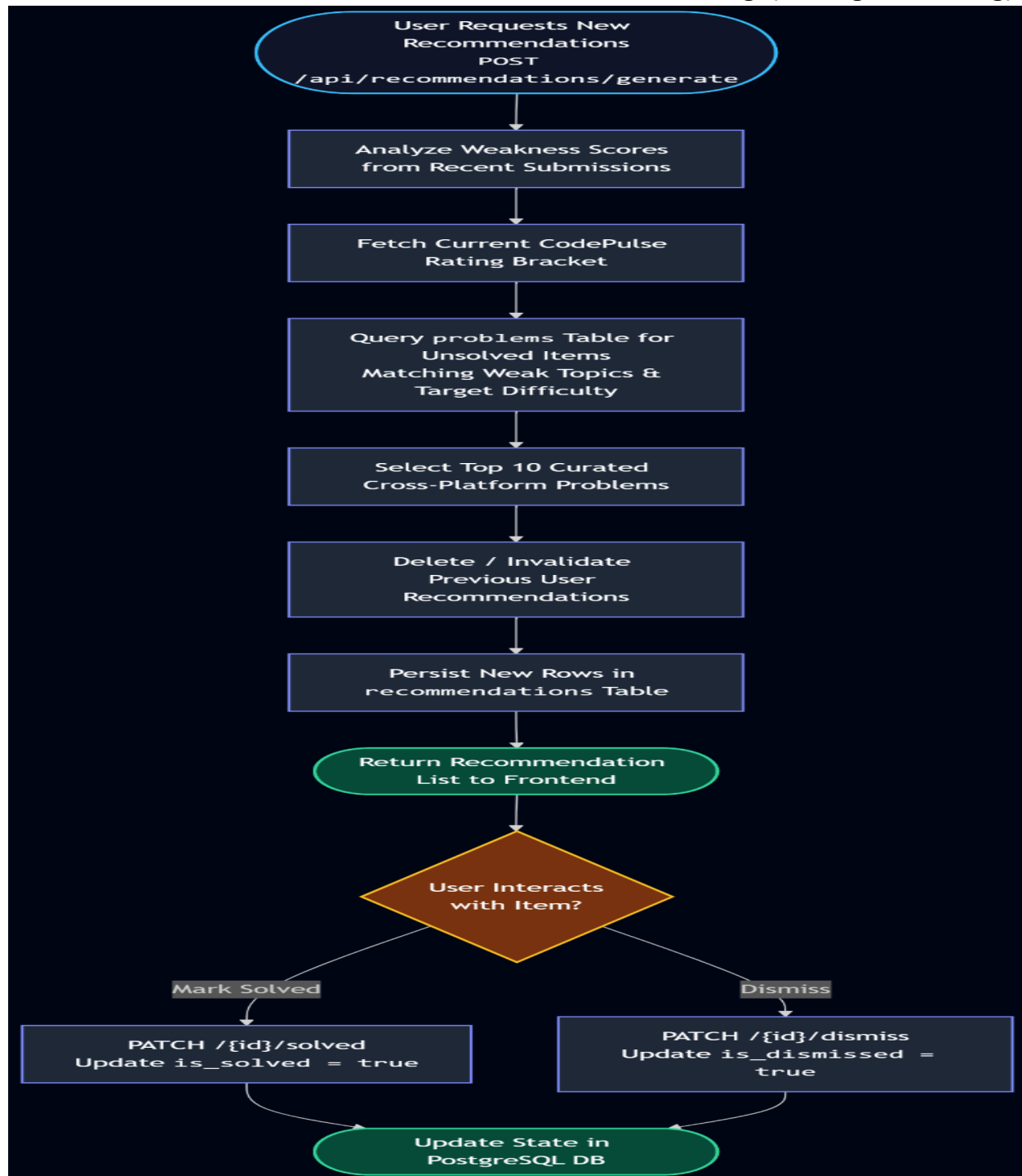


Figure 10 : Activity Diagram —Algorithmic Recommendation & Feedback Pipeline

3.2.5 AD-5: AI Coach Prompt-to-Recommendation Pipeline

This flow captures the full vertical pipeline for context construction, prompt assembly, Mistral AI API execution, `JsonNode` parsing, session persistence, and error handling.

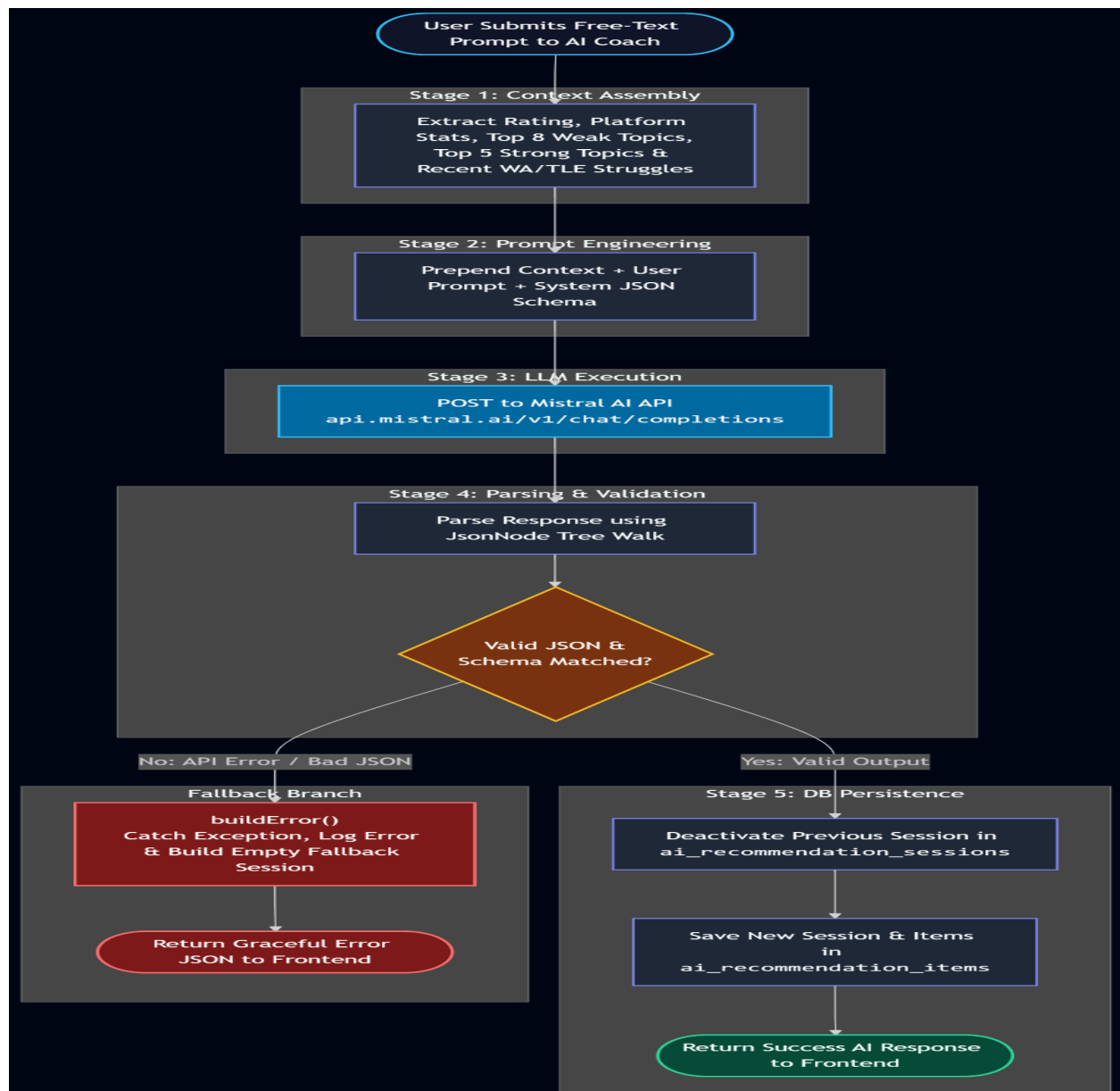


Figure 11 : Activity Diagram —AI Coach Prompt-to-Recommendation Pipeline

4. Data & Information Modeling

4.1 Database Schema

CodePulse uses a PostgreSQL relational database with 7 core tables. All tables use auto-generated bigint primary keys and appropriate foreign key constraints. The schema is managed by Hibernate with ddl-auto=update.

Table	Key Columns	Purpose
users	id, username, email, password_hash, codeforces_handle, leetcode_handle, atcoder_handle, codechef_handle, role, last_synced_at	Platform handles + auth
problems	id, platform_id, platform, title, difficulty_rating, difficulty_label, problem_url	Cross-platform problem catalogue
topics	id, name, slug	Algorithm topic tags
problem_topics	problem_id (FK), topic_id (FK)	Many-to-many join
submissions	id, user_id (FK), problem_id (FK), platform_submission_id, verdict, language, submitted_at	All synced submissions
recommendations	id, user_id (FK), problem_id (FK), reason, score, is_solved, is_dismissed	Algorithmic recs
ai_recommendation_sessions	id, user_id (FK), prompt, coach_insight, focus_areas_json, model_used, active	AI Coach sessions
ai_recommendation_items	id, session_id (FK), title, platform, url, difficulty,	Items within a session

	topics_json, reason, is_solved, is_dismissed	
--	---	--

4.2 ER Diagram

The entity-relationship diagram below illustrates the primary associations. Key cardinalities:

- User 1 — N Submissions
- Problem N — M Topics (via problem_topics)
- Submission N — 1 Problem
- User 1 — N Recommendations, each referencing 1 Problem
- User 1 — N AiRecommendationSessions
- AiRecommendationSession 1 — N AiRecommendationItems

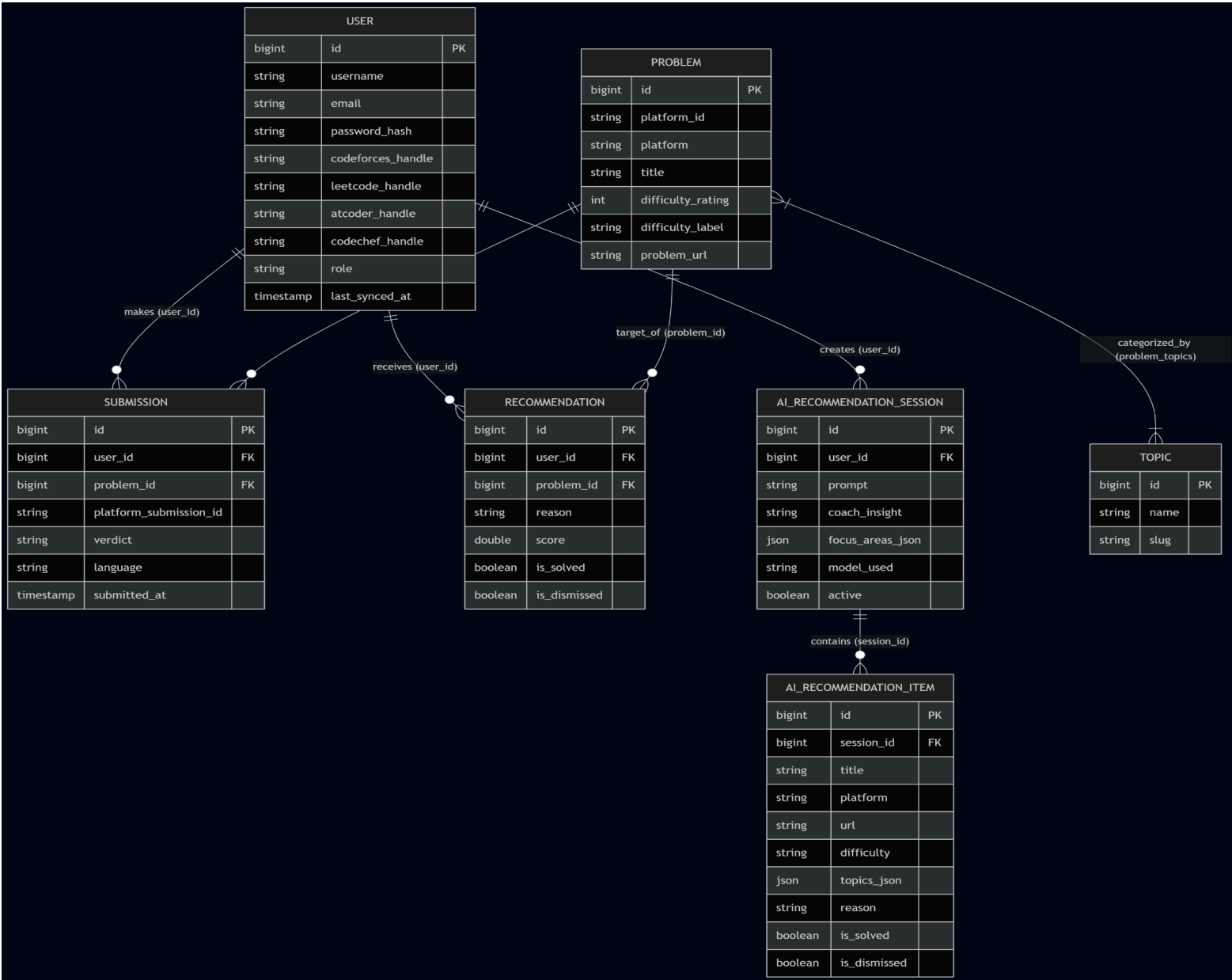


Figure 12 : ER Diagram

5. Software Design

5.1 Component Diagram

The system follows a clean 3-tier architecture with an additional AI integration layer.

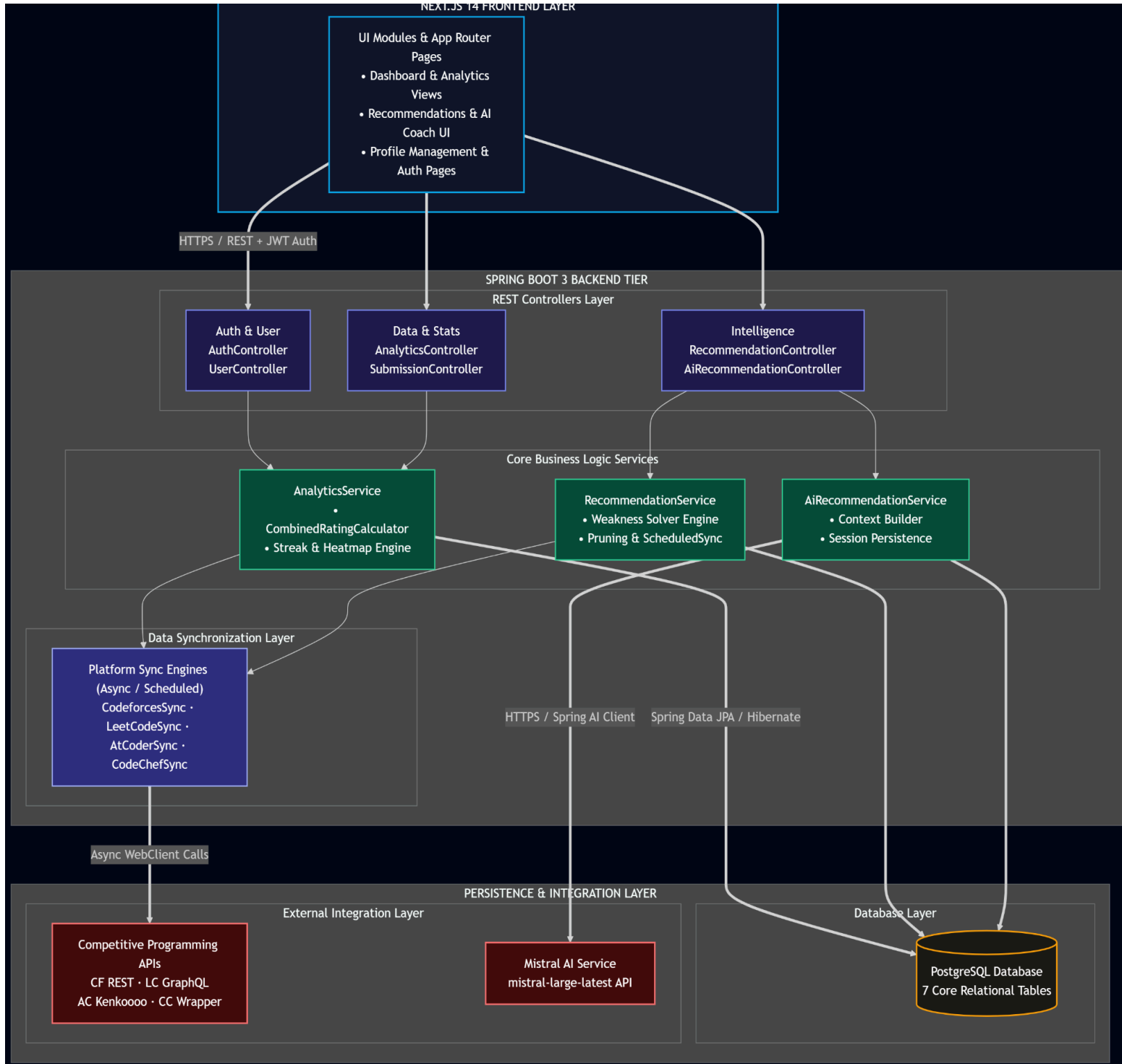


Figure 13 : Component Diagram

5.2 Key Design Decisions

5.2.1 Direct HTTP vs Spring AI SDK for Mistral

Spring AI milestone versions (M4–M6) construct an internal `ObjectMapper` instance that cannot be overridden via any Spring bean customiser. When the Mistral API added a "prompt_tokens_details" field to its response object, every call through the SDK threw `UnrecognizedPropertyException`. The solution adopted in CodePulse is to bypass the SDK entirely and call the Mistral REST endpoint directly via `RestTemplate`, deserialising the response into a `JsonNode` tree. Since `JsonNode` accepts any JSON without a target class, future API schema additions are silently ignored without requiring code changes.

5.2.2 CodePulse Rating Algorithm

A single Elo-like number is computed from all connected platforms:

- CF Rating: actual rating from `/user.rating` or estimated from average difficulty of solved problems.
- LC Rating: $1200 + (\text{easy} \times 2) + (\text{medium} \times 9) + (\text{hard} \times 22) + \text{acceptance-rate bonus}$.
- AC Rating: average difficulty of solved AtCoder problems $\times 1.05$.
- CC Rating: $1000 + (\text{easy} \times 3) + (\text{medium} \times 10) + (\text{hard} \times 20) + (\text{expert} \times 35)$.
- Weight per platform = $\sqrt{(\text{submission count on that platform})}$ — diminishing returns for lopsided activity.
- Final = weighted average, clipped to [800, 3800].

5.2.3 Submission Pruning

To prevent unbounded database growth, each user is capped at 300 submissions (configurable). After every sync, `SubmissionPruningService` checks the count and deletes the oldest entries in a single bulk `DELETE` query. Analytics always reflect the most recent 300 submissions.

5.2.4 Hidden Insights Engine

Rather than showing only raw numbers, the frontend's `InsightsEngine` analyses the analytics response and automatically generates 6–10 contextual findings, categorised as Critical, Warning, Success, Tip, or Pattern. Examples include detecting that a user's failure rate on Dynamic Programming exceeds 60%

(Critical), noticing that 75% of solves are Beginner/Easy level (Warning — comfort zone alert), or recognising a 14-day consecutive streak (Success). These are computed client-side from the API response with no additional backend calls.

6. AI Engineering Design

6.1 AI Pipeline

The AI recommendation pipeline consists of five stages:

- Stage 1 — Context Assembly: The backend constructs a rich Markdown-formatted context string including the user's CodePulse Rating, per-platform stats, top 8 weak topics with failure rates, top 5 strong topics, difficulty distribution, last 15 accepted problem titles, and last 5 WA/TLE struggle topics.
- Stage 2 — Prompt Engineering: The context is prepended to the user's free-text prompt and a strict JSON schema. A system prompt instructs the model to return only valid, complete JSON with no surrounding text.
- Stage 3 — LLM Call: A POST request is made to `api.mistral.ai/v1/chat/completions` using `RestTemplate`. The response is parsed as `JsonNode`, making the pipeline immune to API schema additions.
- Stage 4 — Parsing & Validation: The JSON tree is walked to extract recommendations, coach insight, and focus areas. Malformed responses are caught and logged; the user receives a graceful error message.
- Stage 5 — Persistence: The previous session is deactivated and the new session + items are saved to `ai_recommendation_sessions` and `ai_recommendation_items`. The response is returned immediately; persistence is part of the same transaction.

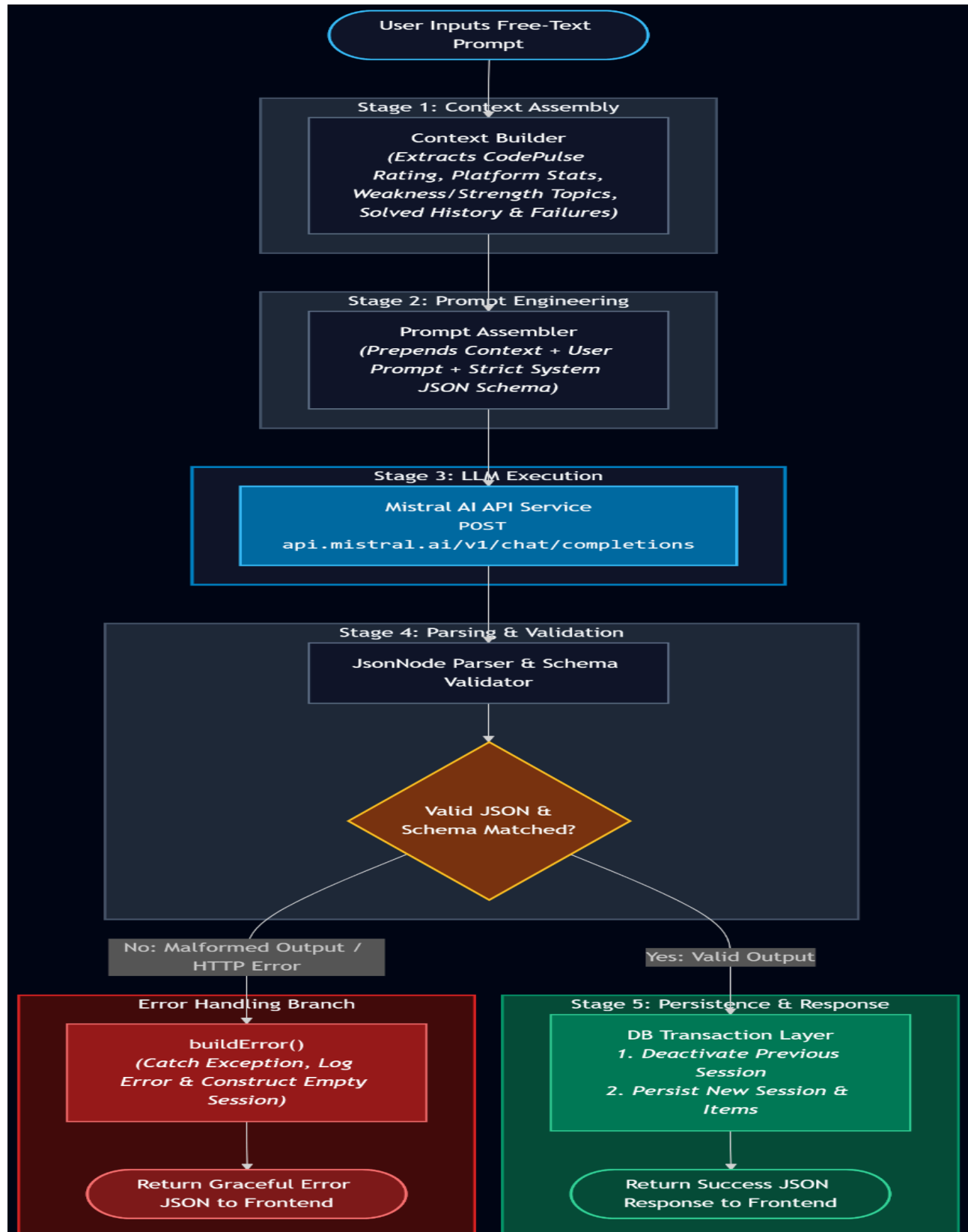


Figure 14 : AI Pipeline Flow Diagram

6.2 Model Selection

Mistral mistral-large-latest was selected for the following reasons:

- Strong knowledge of competitive programming problems across CF, LC, AC, and CC from training data.
- Reliable JSON-only output mode when instructed via system prompt.
- Competitive pricing with a free tier available via console.mistral.ai.
- OpenRouter (openrouter.ai) is provided as a configured fallback, allowing the model to be switched to DeepSeek, Gemini, or any OpenAI-compatible endpoint via a single property change.

6.3 Prompt Design

The system prompt enforces four absolute rules: (1) return only valid complete JSON, (2) never truncate, (3) recommend only real problems with real URLs, and (4) personalise the "reason" field to reference the specific user's weakness or goal. The user message appends a schema template specifying exactly the fields and count expected. This structured prompting approach consistently produces parseable JSON without markdown fences.

7. Preliminary Test Plan

7.1 Testing Objectives

- Validate all 22 functional requirements through unit and integration tests.
- Verify that the CodePulse Rating formula produces results within expected bounds for edge cases (0 platforms connected, one platform only, all 4 connected).
- Confirm that the AI pipeline handles Mistral API errors gracefully without crashing.
- Test submission deduplication and pruning correctness.
- Verify JWT token validation and endpoint protection.

7.2 Features to Be Tested

- Registration and login — correct token returned, password not stored plaintext.
- Profile handle update — new handles reflected on dashboard and dashboard profile fetch.

- Codeforces sync — new submissions saved, existing ones not duplicated.
- LeetCode GraphQL sync — Easy/Medium/Hard labels stored correctly.
- Pruning — after exceeding 300, oldest are removed, count returns to 300.
- Analytics — combined streak calculation correct across 2 platforms.
- Recommendation generation — only unsolved problems recommended.
- AI Coach — prompt submitted, session created, previous session deactivated.
- Mistral error handling — network failure returns graceful error, not stack trace.
- JWT protection — unauthenticated request to `/api/analytics/me` returns 401.

7.3 Test Cases

TC #	Test Case	Input	Expected Output	Priority
TC-01	Register with valid data	POST <code>/api/auth/register</code> with unique email/username	201 + JWT token returned	High
TC-02	Register duplicate email	Same email twice	400 "Email already registered"	High
TC-03	Login with wrong password	Correct email, wrong password	401 "Invalid credentials"	High
TC-04	Access analytics without token	GET <code>/api/analytics/me</code> , no Auth header	401 Unauthorized	High
TC-05	Save CF handle	PUT <code>/api/users/me</code> with <code>codeforcesHandle</code>	200, handle visible in GET <code>/me</code> response	High
TC-06	CF Sync deduplication	Sync called twice, same submissions	Submission count	High

			unchanged after 2nd sync	
TC-07	Pruning at 300 limit	Insert 310 submissions for user	Submission count = 300, oldest 10 removed	High
TC-08	Analytics — zero platforms	No handles set, GET /api/analytics/me	200, all zeros, empty platform breakdown	Medium
TC-09	CodePulse Rating bounds	1 user with only 1 easy LC problem solved	Rating in range [800, 1250]	Medium
TC-10	Curated recs generation	POST /api/recommendations/generate	200, ≤ 10 unsolved problems returned	High
TC-11	Mark recommendation solved	PATCH /api/recommendations/{id}/solved	200, item no longer in active list	Medium
TC-12	AI prompt submission	POST /api/ai/recommend with valid prompt	200, session with ≥ 1 recommendation	High
TC-13	AI session persistence	GET /api/ai/sessions/latest after TC-12	Same session returned without regeneration	High
TC-14	AI regeneration deactivates old	Submit 2nd prompt for same user	Only new session is active; old is inactive	High

TC-15	Mistral API error handling	Invalid API key configured	200 with empty recs + error insight message	High
TC-16	Streak across 2 platforms	3 CF subs on day 1, 2 LC subs on day 2	currentStreak = 2	Medium
TC-17	Topic weakness ranking	10 DP attempts, 8 fail; 5 graph, 2 fail	DP weakness > graph weakness	Medium
TC-18	Admin list users	GET /api/admin/users with ADMIN token	200, list of all users	Low
TC-19	Admin endpoint with USER role	GET /api/admin/users with USER token	403 Forbidden	High
TC-20	Recent submissions order	GET /api/submissions/recent?limit=5	5 most recent, newest first	Medium

8. Timeline

8.1 Development Phases

Phase	Activities	Duration	Status
Phase 1	Requirements, SRS, database schema design, entity modelling	Weeks 1–2	Complete
Phase 2	Auth module, JWT security, User entity, profile API	Weeks 3–4	Complete
Phase 3	Codeforces + LeetCode sync services, Problem/Topic/Submission entities	Weeks 5–6	Complete
Phase 4	AtCoder + CodeChef sync, combined analytics service, CodePulse Rating algorithm	Weeks 7–8	Complete

Phase 5	Recommendation engine, pruning service, scheduled sync	Week 9	Complete
Phase 6	AI Coach integration (Mistral direct HTTP), session persistence, context builder	Weeks 10–11	Complete
Phase 7	Next.js frontend: Dashboard, Analytics, Recommendations, AI Coach, Profile	Weeks 12–14	Complete
Phase 8	Integration testing, bug fixes, hidden insights engine, report writing	Weeks 15–16	In Progress

8.2 Gantt Chart

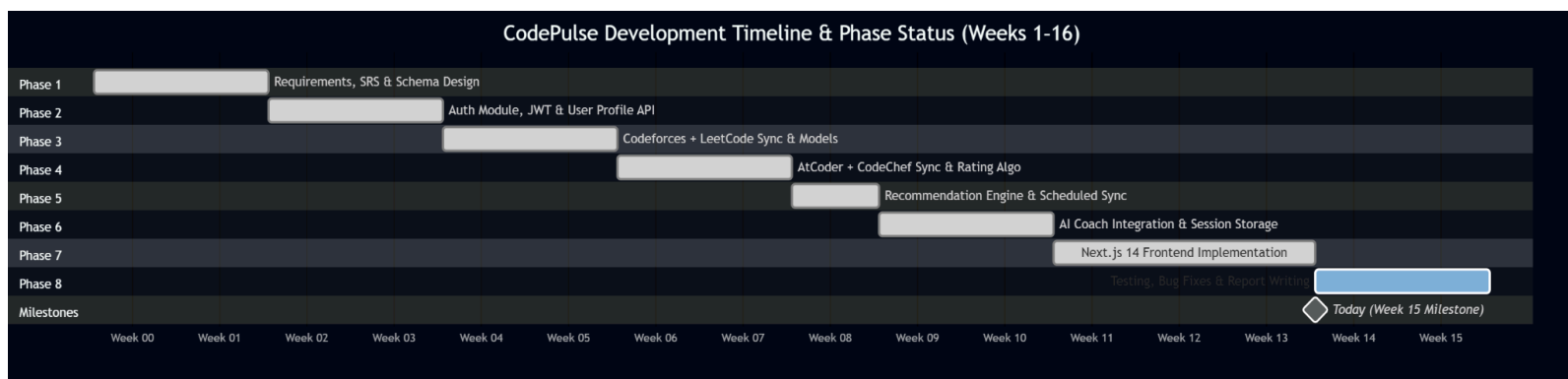


Figure 15 : Gantt Chart (Weeks 1–16)

9. Appendix

9.A API Endpoint Reference

Method	Endpoint	Description	Auth
POST	/api/auth/register	Create account	Public
POST	/api/auth/login	Authenticate, receive JWT	Public
GET	/api/users/me	Get own profile + handles	JWT
PUT	/api/users/me	Update profile / handles	JWT

GET	/api/analytics/me	Full combined analytics	JWT
POST	/api/analytics/sync	Trigger multi-platform sync	JWT
GET	/api/submissions/recent	Recent submissions (all platforms)	JWT
GET	/api/recommendations	Active curated recommendations	JWT
POST	/api/recommendations/generate	Generate new recommendations	JWT
PATCH	/api/recommendations/{id}/solved	Mark solved	JWT
PATCH	/api/recommendations/{id}/dismiss	Dismiss recommendation	JWT
POST	/api/ai/recommend	AI Coach: generate from prompt	JWT
GET	/api/ai/sessions/latest	Load stored AI session	JWT
PATCH	/api/ai/items/{id}/solved	Mark AI item solved	JWT
PATCH	/api/ai/items/{id}/dismiss	Dismiss AI item	JWT
GET	/api/admin/users	List all users (admin only)	JWT + ADMIN

9.B Project Source Code Link

<https://github.com/Badhon1401/SPL-3>